

Módulo 7 · Programación orientada a objetos

Clases, objetos, atributos, métodos, constructores, encapsulación y relaciones entre objetos.

De programas sueltos a modelos del mundo real

La POO ayuda a organizar el código alrededor de entidades: Producto, CuentaBancaria, Alumno o Pedido. Cada entidad agrupa datos y comportamientos relacionados.

- Definir clases y crear objetos.
- Encapsular atributos para proteger los datos.
- Usar constructores, getters y setters.
- Representar relaciones simples entre objetos.

Resultados de aprendizaje

Qué debería saber hacer el alumnado al terminar el módulo.

- Explicar la diferencia entre clase y objeto con ejemplos sencillos.
- Crear clases con atributos y métodos.
- Construir objetos con new y usar sus métodos.
- Aplicar private para proteger atributos.
- Crear constructores para inicializar objetos correctamente.
- Usar getters, setters y toString de forma razonada.
- Modelar problemas sencillos como productos, cuentas o pedidos.

Objetivo docente: que dejen de escribir todo en main y empiecen a repartir responsabilidades.

¿Por qué aparece la POO?

Hasta ahora hemos trabajado con variables, bucles, métodos y colecciones. Falta organizar mejor los datos.

Sin POO

- Variables sueltas en main
- Métodos aislados
- Difícil ver qué dato pertenece a qué entidad
- Código más difícil de mantener
- Aumenta el riesgo de repetir lógica

Con POO

- Agrupamos datos y comportamiento
- Cada clase representa una idea
- El código se entiende mejor
- Podemos reutilizar objetos
- Es más fácil ampliar el programa

Idea principal de la POO

Un programa se construye con objetos que colaboran entre sí.

Producto

nombre
precio
stock

Pedido

cliente
líneas
fecha

Cuenta

titular
saldo

- Cada objeto tiene datos propios.
- Cada objeto ofrece operaciones para trabajar con esos datos.
- El programa se reparte entre piezas más pequeñas.

Clase y objeto

Son conceptos relacionados, pero no significan lo mismo.

Clase

- Es el molde o plantilla.
- Define qué datos tendrá un objeto.
- Define qué acciones podrá realizar.
- Ejemplo: `class Producto`.

Objeto

- Es un elemento concreto creado desde la clase.
- Tiene valores propios.
- Podemos crear muchos objetos de la misma clase.
- Ejemplo: un producto llamado Teclado.

Analogía sencilla

La clase es el plano; el objeto es la casa construida.

CLASE

```
public class Producto {  
    String nombre;  
    double precio;  
}
```

OBJETOS

```
Producto p1 = new Producto();  
Producto p2 = new Producto();
```

- La clase describe cómo serán los objetos.
- Los objetos existen cuando usamos new.
- Cada objeto puede guardar valores diferentes.

Primera clase: Producto

Una clase sencilla puede empezar con atributos, pero luego la mejoraremos.

```
public class Producto {  
    String nombre;  
    double precio;  
    int stock;  
}
```

- Producto es el nombre de la clase.
- nombre, precio y stock son atributos.
- Los atributos describen el estado del producto.
- Todavía no hay métodos ni protección de datos.

Este primer paso sirve para entender la estructura. Después usaremos private y constructores.

Atributos

Los atributos son variables que pertenecen a una clase.

```
public class Producto {  
    String nombre;  
    double precio;  
    int stock;  
}
```

- Se declaran dentro de la clase, pero fuera de main.
- Representan características del objeto.
- Cada objeto tiene su propia copia de esos valores.
- Deben tener nombres claros y relacionados con el modelo.

No es lo mismo una variable local que un atributo: la variable local vive dentro de un método; el atributo pertenece al objeto.

Crear objetos con new

Para usar una clase necesitamos crear objetos.

```
public class Principal {  
    public static void main(String[] args) {  
        Producto producto1 = new Producto();  
        producto1.nombre = "Teclado";  
        producto1.precio = 29.99;  
        producto1.stock = 15;  
    }  
}
```

- new Producto() crea un objeto nuevo.
- producto1 es la variable que referencia ese objeto.
- Con el punto accedemos a sus atributos.
- Este acceso directo funciona, pero no es lo más recomendable.

El operador punto

El punto permite acceder a atributos y métodos de un objeto.

```
producto1.nombre = "Teclado";  
producto1.precio = 29.99;  
producto1.stock = 15;  
  
System.out.println(producto1.nombre);
```

- objeto.atributo permite leer o modificar un dato.
- objeto.metodo() permite ejecutar una acción.
- El punto se lee como: “dentro de este objeto”.

Más adelante evitaremos modificar atributos directamente para proteger el estado interno.

Métodos dentro de una clase

Un método representa una acción o cálculo que puede realizar el objeto.

```
public class Producto {  
    String nombre;  
    double precio;  
    int stock;  
  
    double valorInventario() {  
        return precio * stock;  
    }  
}
```

- valorInventario calcula precio por stock.
- El método usa los atributos del propio objeto.
- No necesita recibir precio y stock como parámetros porque ya pertenecen al objeto.
- El nombre del método debe expresar una acción o cálculo.

Usar un método de objeto

Una vez creado el objeto, podemos llamar a sus métodos con el punto.

```
Producto p = new Producto();  
p.nombre = "Teclado";  
p.precio = 29.99;  
p.stock = 15;  
  
double valor = p.valorInventario();  
System.out.println(valor);
```

- `p.valorInventario()` ejecuta el método en ese producto concreto.
- Si otro producto tiene otro precio o stock, el resultado será diferente.
- La lógica queda dentro de la clase `Producto`, no dispersa por `main`.

El problema del acceso directo

Si los atributos quedan accesibles, cualquier parte del programa puede poner valores incorrectos.

```
Producto p = new Producto();  
p.nombre = "Ratón";  
p.precio = -50;  
p.stock = -10;
```

- Un precio negativo no tiene sentido.
- Un stock negativo puede romper los cálculos.
- La clase debería proteger sus propios datos.
- Aquí aparece la idea de encapsulación.

Pregunta para el alumnado: ¿debería poder escribirse cualquier valor en cualquier atributo?

Encapsulación

Encapsular significa proteger el estado interno del objeto.

```
public class Producto {  
    private String nombre;  
    private double precio;  
    private int stock;  
}
```

- private impide acceder directamente desde fuera de la clase.
- El objeto controla cómo se modifican sus datos.
- Las operaciones válidas se ofrecen mediante métodos públicos.
- Es una de las ideas principales de la POO.

El manual recomienda encapsular atributos y ofrecer operaciones válidas mediante métodos públicos.

private y public

Los modificadores de acceso indican desde dónde se puede usar algo.

private

- Solo se puede usar dentro de la propia clase.
- Suele aplicarse a atributos.
- Protege los datos internos.
- Evita modificaciones sin control.

public

- Se puede usar desde otras clases.
- Suele aplicarse a métodos necesarios.
- Forma la interfaz pública del objeto.
- Permite interactuar de forma segura.

Getters

Un getter permite consultar un atributo privado.

```
public class Producto {  
    private double precio;  
  
    public double getPrecio() {  
        return precio;  
    }  
}
```

- El atributo precio es private.
- getPrecio() es public.
- Devuelve el valor sin permitir modificarlo directamente.
- El nombre suele empezar por get.

Setters

Un setter permite modificar un atributo privado, pero con control.

```
public void setPrecio(double precio) {  
    if (precio < 0) {  
        throw new IllegalArgumentException("Precio  
negativo");  
    }  
    this.precio = precio;  
}
```

- Recibe el nuevo valor como parámetro.
- Comprueba si el valor es válido.
- Si es incorrecto, lanza un error.
- Si es correcto, actualiza el atributo.

El setter no es solo “poner un valor”; es el lugar donde se puede validar.

La palabra this

this representa al objeto actual.

```
private double precio;  
  
public void setPrecio(double precio) {  
    this.precio = precio;  
}
```

- precio sin this es el parámetro recibido.
- this.precio es el atributo del objeto.
- Se usa cuando el atributo y el parámetro tienen el mismo nombre.
- Ayuda a distinguir claramente ambos elementos.

Constructores

Un constructor inicializa el objeto cuando se crea.

```
public class Producto {  
    private String nombre;  
    private double precio;  
    private int stock;  
  
    public Producto(String nombre, double precio, int stock) {  
        this.nombre = nombre;  
        this.precio = precio;  
        this.stock = stock;  
    }  
}
```

- Tiene el mismo nombre que la clase.
- No tiene tipo de retorno, ni siquiera void.
- Se ejecuta al usar new.
- Evita crear objetos a medias.

Crear objetos con constructor

Con un constructor podemos pasar los datos desde el principio.

```
Producto p1 = new Producto("Teclado", 29.99, 15);  
Producto p2 = new Producto("Ratón", 12.50, 30);  
  
System.out.println(p1.getPrecio());  
System.out.println(p2.getPrecio());
```

- Cada llamada a new crea un objeto independiente.
- Los valores entre paréntesis se envían al constructor.
- El objeto nace con datos iniciales.
- Así se reduce el riesgo de olvidar atributos.

Constructor con validación

Es mejor validar desde el momento en que nace el objeto.

```
public Producto(String nombre, double precio,
int stock) {
    this.nombre = nombre;
    setPrecio(precio);
    setStock(stock);
}
```

- El constructor reutiliza los setters.
- Así no duplicamos validaciones.
- Si el precio o el stock son incorrectos, el objeto no se crea bien.
- La clase protege su coherencia desde el inicio.

Buena práctica: no permitir que exista un objeto con datos absurdos.

Método toString

toString define cómo se representa un objeto como texto.

```
@Override
public String toString() {
    return nombre + " - " + precio + " EUR - stock: "
+ stock;
}
```

- Sirve para mostrar objetos de forma legible.
- Evita imprimir referencias poco útiles.
- Es muy práctico al listar productos o elementos de una colección.
- @Override indica que estamos redefiniendo un método heredado.

Clase Producto completa

Versión más completa: atributos privados, constructores, validación de String

```
public class Producto {  
    private String nombre;  
    private double precio;  
    private int stock;  
  
    public Producto(String nombre, double precio, int stock) {  
        this.nombre = nombre;  
        setPrecio(precio);  
        this.stock = stock;  
    }  
  
    public double getPrecio() { return precio; }  
  
    public void setPrecio(double precio) {  
        if (precio < 0) throw new IllegalArgumentException("Precio negativo");  
        this.precio = precio;  
    }  
}
```

- Esta estructura es la base del módulo.
- Separa datos, inicialización, consulta, modificación y representación.
- A partir de aquí podemos crear objetos más seguros y mantenibles.

Responsabilidades de una clase

Una clase no debería hacerlo todo.

- Producto representa datos y reglas de un producto.
- CuentaBancaria representa saldo y operaciones bancarias.
- Pedido representa una compra o solicitud.
- Una clase debe tener una responsabilidad clara.
- Si una clase crece demasiado, quizá haya que dividirla.

Idea clave: una clase buena se entiende por su nombre. Si el nombre es confuso, probablemente el diseño también lo sea.

Variables locales y atributos

No todas las variables viven en el mismo lugar.

Atributos

- Están dentro de la clase.
- Representan el estado del objeto.
- Existen mientras existe el objeto.
- Ejemplo: nombre, precio, stock.

Variables locales

- Están dentro de un método.
- Sirven para cálculos temporales.
- Solo existen durante la ejecución del método.
- Ejemplo: total, descuento, resultado.

Ejemplo: método valorInventario

Un método puede usar atributos y devolver un resultado calculado.

```
public double valorInventario() {  
    return precio * stock;  
}
```

- No modifica el objeto.
- Solo calcula y devuelve un valor.
- Es fácil de probar.
- El nombre explica claramente qué calcula.

Métodos pequeños y claros facilitan la depuración y reducen errores.

Relaciones entre objetos

Un objeto puede contener o usar otros objetos.

Producto

LineaPedido

Pedido

- LineaPedido puede contener un Producto y una cantidad.
- Pedido puede contener varias líneas de pedido.
- Así modelamos problemas reales con piezas relacionadas.

Composición: LineaPedido

Una línea de pedido une un producto con una cantidad.

```
public class LineaPedido {  
    private Producto producto;  
    private int cantidad;  
  
    public double subtotal() {  
        return producto.getPrecio() * cantidad;  
    }  
}
```

- producto es un atributo de tipo Producto.
- cantidad indica cuántas unidades se compran.
- subtotal usa datos del producto y de la línea.
- Esta clase tiene una responsabilidad concreta.

Ejercicio 13: Cuenta bancaria

Modelar una cuenta con titular y saldo.

```
/*  
Enunciado:  
Modela una CuentaBancaria con titular y saldo.  
Incluye ingresar, retirar y consultar saldo.  
No permitas retirar más saldo del disponible.  
*/
```

- Atributos: titular y saldo.
- Constructor: recibe titular y saldo inicial.
- Método ingresar: aumenta el saldo.
- Método retirar: comprueba que hay saldo suficiente.
- Método consultarSaldo: devuelve el saldo actual.

Este ejercicio es ideal para practicar encapsulación y validación dentro de métodos.

Diseño de CuentaBancaria

Antes de programar conviene decidir qué tendrá la clase.

Datos

- titular: String
- saldo: double
- Ambos deben ser private
- El saldo no debería modificarse directamente

Operaciones

- ingresar(double cantidad)
- retirar(double cantidad)
- getSaldo()
- toString() para mostrar la cuenta

Ejercicio 14: Pedido

Modelar productos y líneas de pedido.

```
/*  
Enunciado:  
Crea Producto y LineaPedido.  
Una línea contiene un producto y una cantidad  
y calcula su subtotal.  
*/
```

- Clase Producto: nombre, precio y stock.
- Clase LineaPedido: producto y cantidad.
- Método subtotal: precio por cantidad.
- Puede ampliarse con Pedido y una lista de líneas.

Aquí aparece una relación entre objetos: una LineaPedido usa un Producto.

Errores frecuentes en POO

Problemas típicos cuando se empieza con clases y objetos.

- Olvidar crear el objeto con new antes de usarlo.
- Poner todo en main y no repartir responsabilidades.
- Hacer atributos public sin necesidad.
- Crear setters sin validación cuando los datos tienen límites.
- Confundir el nombre del archivo con el nombre de la clase pública.
- Olvidar this cuando atributo y parámetro se llaman igual.

Para corregir: primero revisa el modelo y la lógica; después revisa estilo y nombres.

Buenas prácticas

Hábitos que conviene exigir desde el principio.

- Usar nombres descriptivos: Producto, CuentaBancaria, LineaPedido.
- Mantener atributos private.
- Validar datos importantes en setters o constructores.
- Crear métodos pequeños con una responsabilidad clara.
- Evitar repetir código si puede estar en un método.
- Probar valores normales, extremos y erróneos.

```
public double valorInventario() {  
    return precio * stock;  
}
```

Mini práctica guiada

Crear un producto, consultar su precio y calcular el valor del inventario.

- Paso 1: crear la clase Producto.
- Paso 2: añadir atributos privados.
- Paso 3: crear constructor.
- Paso 4: crear getPrecio y setPrecio con validación.
- Paso 5: crear valorInventario.
- Paso 6: probar desde una clase Principal.

```
Producto p = new Producto("Teclado", 29.99, 15);  
System.out.println(p);  
System.out.println(p.valorInventario());
```

La prueba debe demostrar que el objeto se crea, se muestra y calcula correctamente.

Actividad de cierre

Preguntas para comprobar comprensión antes de avanzar.

- ¿Qué diferencia hay entre clase y objeto?
- ¿Por qué los atributos deberían ser private?
- ¿Qué hace un constructor?
- ¿Cuándo usamos this?
- ¿Qué ventaja tiene toString?
- ¿Qué clase crearías para representar un alumno? ¿Qué atributos tendría?

No avanzar si el alumnado solo copia el código: debe poder explicar qué hace cada parte.

Cierre del módulo

La POO es el puente entre código básico y aplicaciones mejor organizadas.

- Una clase define un modelo.
- Un objeto es una instancia concreta de ese modelo.
- Los atributos guardan estado.
- Los métodos definen comportamiento.
- La encapsulación protege los datos.
- Las relaciones entre objetos permiten modelar problemas reales.

Siguiente paso natural: herencia, interfaces y excepciones, pero solo cuando clases, objetos y encapsulación estén claros.